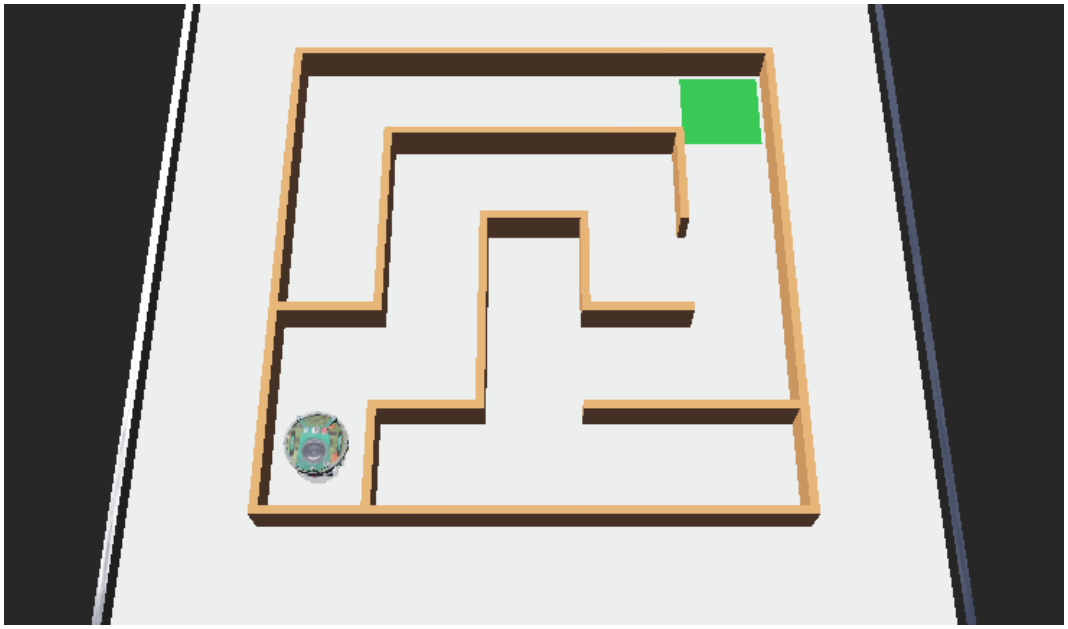
Zadanie 3: Labirynt



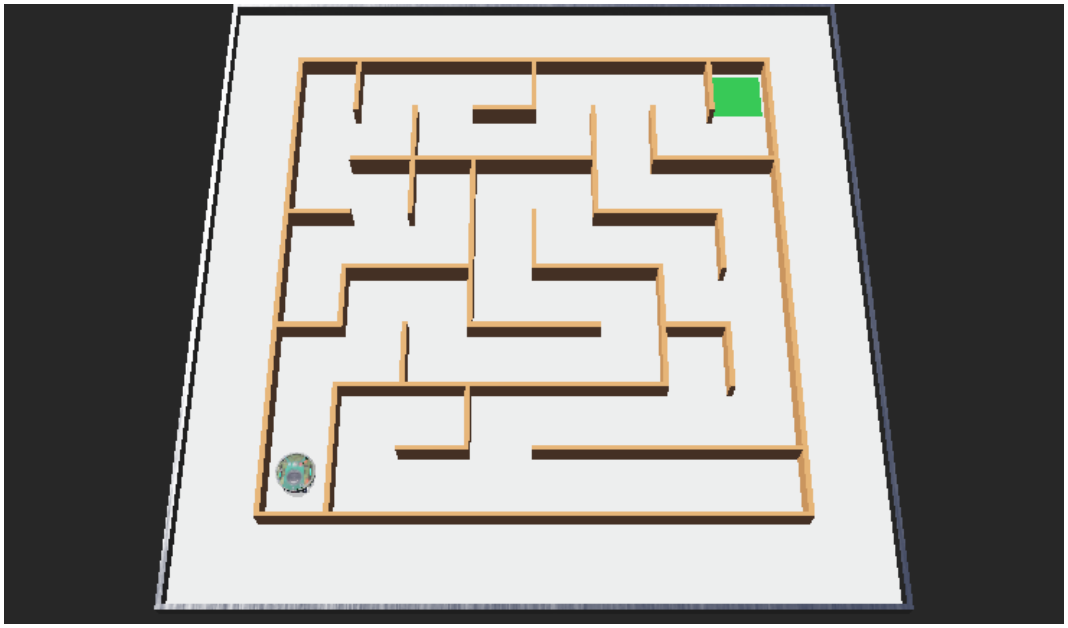
Opis zadania

Robot e-puck startuje w rogu labiryntu. Napisz program, który doprowadzi go do **zielonego pola celu** jak najszybciej i bez dotykania ścian.

Przygotowaliśmy dwa labirynty:

- **Labirynt 1 (łatwy)**, plik `worlds/maze_1.wbt`: 5 × 5 pól.
- **Labirynt 2 (trudny)**, plik `worlds/maze_2.wbt`: 8 × 8 pól.

Ten sam program powinien przejść oba labirynty bez zmian.



Struktura folderu

```
03_maze/
```

```
├── worlds/
│   ├── maze_1.wbt
│   └── maze_2.wbt
└── controllers/
    ├── maze_solver/maze_solver.py   TWÓJ PROGRAM
    └── judge/                       sędzia (nie modyfikuj)
```

Robot i labirynt

- Pola labiryntu mają 12 × 12 cm, ściany mają 1 cm grubości i 5 cm wysokości. Robot ma średnicę ok. 7 cm, więc z każdej strony zostaje mniej niż 2 cm zapasu.
- `left wheel motor`, `right wheel motor`: silniki kół, maksymalnie 6,28 rad/s.
- `left wheel sensor`, `right wheel sensor`: enkodery kół (obrót koła w radianach). Promień koła to 2,05 cm, rozstaw kół 5,2 cm. W symulacji warto te wartości skalibrować, bo robot obraca się trochę mniej, niż wynika z obliczeń.
- `ps0` … `ps7`: czujniki odległości (podczerwień) dookoła robota. `ps0` i `ps7` patrzą do przodu, `ps2` w prawo, `ps5` w lewo, `ps1` i `ps6` na skos do przodu, `ps3` i `ps4` do tyłu.
- Orientacyjne odczyty, gdy robot stoi na środku pola: ok. **65–75 bez ściany**, ok. **150–170 ze ścianą z boku**, ok. **110–140 ze ścianą z przodu**. Im bliżej ściany, tym większa wartość. Czujniki mają szum.
- `gs0`, `gs1`, `gs2`: czujniki podłoża. Zielone pole celu jest ciemniejsze od podłogi (odczyt spada poniżej ok. 600), więc robot może rozpoznać, że dojechał.

Zasady i ocena

- Sędzia mierzy czas od startu do chwili, gdy środek robota znajdzie się na zielonym polu.
- Każde dotknięcie ściany dodaje **2 s kary**.
- Limit czasu: 180 s dla labiryntu 1 i 300 s dla labiryntu 2.
- Wynik to czas plus kary. Najpierw liczy się liczba przejechanych labiryntów, potem suma wyników.

Poziomy:

- **podstawowy**: przejazd labiryntu 1,
- **pełny**: przejazd obu labiryntów,
- **dla chętnych**: w labiryncie 2 metoda „trzymaj się prawej (lub lewej) ściany” wymaga przejechania 62 pól, a najkrótsza droga ma tylko 30. Spróbuj znaleźć szybszy sposób, np. mapując labirynt podczas jazdy i planując trasę (algorytm flood fill, BFS). Dla porównania nasze rozwiązanie wzorcowe, które trzyma się ściany, przejeżdża labirynt 1 w ok. 58 s i labirynt 2 w ok. 135 s.

Wskazówki

- Zacznij od wypisania wartości czujników w konsoli i sprawdź, jak wyglądają przy ścianie i bez niej.
- Najprostsza strategia to reguła prawej ręki: jeśli po prawej nie ma ściany, skręć w prawo; jeśli jest ściana z przodu, skręć w lewo; w ślepym zaułku zawróć.
- Jazda „pole po polu” (dokładnie 12 cm do przodu, obrót dokładnie o 90°) jest zwykle pewniejsza niż jazda ciągła. Do odmierzania odległości i kątów użyj enkoderów.
- Błędy kierunku się sumują. Wykorzystaj ściany do korekty: utrzymuj równą odległość od bocznych ścian, a stojąc przed ścianą wyrównaj się do niej przednimi czujnikami.
- Odczyty czujników są zaszumione, więc warto uśrednić kilka pomiarów przed podjęciem decyzji.

Co oddać

- plik sterownika (`.py`) z Twoim rozwiązaniem,
- krótkie nagranie ekranu (do 2 minut) pokazujące przejazd/działanie robota i wynik sędziego w konsoli,
- kilka zdań opisu: jak działa Twoje rozwiązanie, co sprawiało trudność i co byś poprawił(a), mając więcej czasu.

Rozwiązanie wyślij w sposób podany przez organizatorów rekrutacji. Możesz korzystać z dokumentacji, internetu i narzędzi AI, ale na rozmowie poprosimy Cię o wyjaśnienie swojego kodu, więc upewnij się, że rozumiesz każdą jego linijkę.

Nie modyfikuj plików sędziego ani świata (poza eksperymentami na własny użytek). Przy ocenie uruchomimy Twój sterownik w oryginalnych światach.

Webots: co to jest i jak zacząć

Czym jest Webots

Webots to darmowy, otwartoźródłowy symulator robotów rozwijany przez firmę Cyberbotics. Pozwala zbudować wirtualny świat z robotami, czujnikami i obiektami, a potem sterować robotami programami (tzw. kontrolerami) napisanymi m.in. w Pythonie, C lub C++. Symulacja uwzględnia fizykę: tarcie, masy, zderzenia i szum czujników, więc program, który działa w Webots, zachowuje się podobnie jak na prawdziwym robocie. Dzięki temu możesz uczyć się robotyki bez sprzętu.

Instalacja

1. Pobierz **Webots R2025a** ze strony https://cyberbotics.com (przycisk Download) albo z https://github.com/cyberbotics/webots/releases. Zadania przygotowano i przetestowano w tej wersji.
2. Zainstaluj program jak zwykłą aplikację (Windows: instalator `.exe`, macOS: plik `.dmg`, Linux: pakiet `.deb` lub `snap`).
3. Zainstaluj **Python 3** (https://www.python.org). Na Windows zaznacz w instalatorze opcję „Add Python to PATH”.
4. Jeśli Webots nie znajdzie Pythona sam, wskaż go w *Tools* → *Preferences* → *General* → *Python command* (np. `python` albo pełna ścieżka do `python.exe`).

Pierwsze uruchomienie zadania

1. Pobierz to repozytorium (`git clone` albo *Code* → *Download ZIP*) i rozpakuj je w zwykłym folderze, np. `C:\robotyka\`. **Nie** umieszczaj go w folderze instalacji Webots (`C:\Program Files\Webots`) ani w folderze synchronizowanym z OneDrive.
2. W Webots wybierz *File* → *Open World* i otwórz plik `.wbt` z folderu `worlds` wybranego zadania.
3. Przy pierwszym otwarciu Webots pobiera modele robotów z internetu, więc potrzebne jest połączenie z siecią. Może to potrwać chwilę.
4. Symulację uruchamiasz przyciskiem play (▶), zatrzymujesz przyciskiem pauzy, a przycisk reset (strzałka zawracająca) przywraca stan początkowy.
5. Komunikaty programów, w tym wynik od sędziego, pojawiają się w konsoli na dole okna.

Jak edytować program robota

Kod sterownika znajduje się w folderze `controllers/<nazwa>/<nazwa>.py`. Możesz go edytować we wbudowanym edytorze Webots (prawy klik na robota → *Edit controller*) albo w dowolnym edytorze, np. VS Code. Po zapisaniu zmian zresetuj symulację, żeby program uruchomił się od nowa.

Przydatne źródła:

- dokumentacja Webots: https://cyberbotics.com/doc/guide/index
- opis funkcji (API) dla Pythona: https://cyberbotics.com/doc/reference/index (przełącz język przykładów na Python)
- tutoriale dla początkujących: https://cyberbotics.com/doc/guide/tutorials

Typowe problemy

- **„Skipped unknown … node or PROTO”** lub **„changed by fallback mechanism”**: Webots nie może pobrać modeli. Sprawdź połączenie z internetem i to, czy projekt nie leży w folderze instalacji Webots.
- **Sterownik nie startuje, błąd Pythona**: sprawdź ustawienie *Python command* w preferencjach.
- **Zmiany w kodzie nie działają**: zapisz plik i zresetuj symulację.
- **Symulacja działa wolno**: wyłącz cienie i antyaliasing w *Tools* → *Preferences* → *OpenGL* albo użyj trybu szybkiego (przycisk z podwójną strzałką, *Fast*).